Data Science I

Version: October 10, 2025

Project Report: Predicting Energy Prices

Submission Deadline: January 25 2026, 21:00 UTC

University of Oldenburg
Winter 2025/2026
Instructors: Jannik Schröder, Wolfram "Wolle" Wingerath

Submitted by: Kryvtsova Vlada, Winkler David, Brüning Menko

Introduction

Electricity markets require generators, traders, and large consumers to plan production and consumption ahead of time. In Europe, one of the most important reference markets is the **day-ahead market**, where participants submit bids and offers for each delivery hour of the next day. The market is then cleared so that supply meets demand, resulting in a separate hourly price (€/MWh) for each hour of the following day. These prices can vary strongly due to demand patterns, renewable generation, fuel costs, and seasonal effects, and they may even become negative during periods of high renewable supply and low demand.

The goal of this project is to forecast Germany's day-ahead electricity prices for a specific target date, **January 27, 2026**, using historical data from 2023–2026. We frame the task as an **hourly supervised forecasting problem** with the target variable being the price for delivery hour *t*. As explanatory variables, we use a compact set of drivers available in our dataset. Some of them are the previous day's price, forecasts for load, wind and solar generation, calendar variables.

To assess the value of increasing model complexity, we implement and compare four levels of models:

- **Level 0**: naive persistence baseline ("tomorrow equals yesterday")
- **Level 1**: seasonal time-series model (SARIMA)
- **Level 2**: linear regression model using multiple features
- **Level 3**: machine-learning model (XGBoost).

Models are evaluated with time-aware splits and standard error metrics to ensure a fair comparison without information leakage. The report presents the dataset, modeling

choices, evaluation methodology, and results, and concludes with limitations and potential improvements for future work.

Part 1. Gathering Data Sources

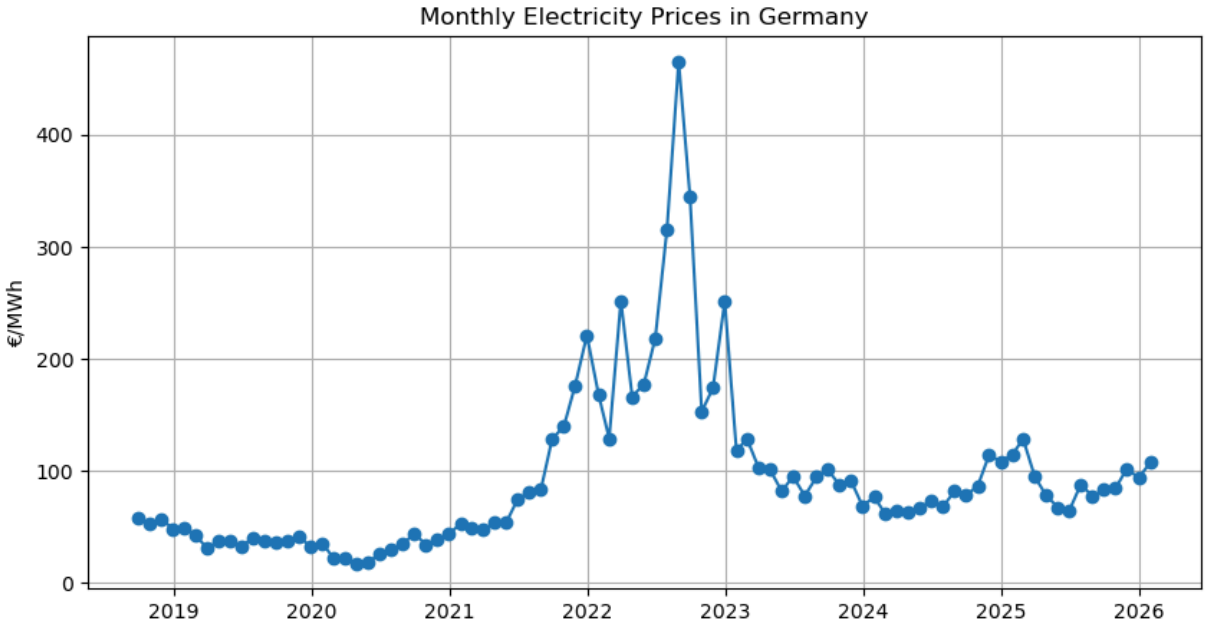
1.1 Time span

The dataset used in this project combines several time series that are relevant for forecasting German day-ahead electricity prices. The data covers the period **from January 1, 2023 to January 25, 2026** with an hourly resolution. This time span is chosen to focus on a period of relatively stable market conditions.

As shown in the figure below, electricity prices in 2022 experienced extreme spikes. These spikes were caused by a geopolitical shock following Russia's reduction of natural gas supplies to Europe. While these events are historically important, they represent exceptional market conditions that are not well explained by the variables used in this project.

By starting the analysis in 2023, the models are trained on data that better reflects typical price dynamics in the German day-ahead electricity market. This leads to lower model prediction errors. The selected time span is still sufficiently long to allow robust training and evaluation, while avoiding distortions caused by rare crisis events.

In [1]:
```
import matplotlib.pyplot as plt
import pandas as pd
import pandas as pd

electricity_prices = pd.read_csv("entsoe_forecasts.csv")
electricity_prices["Date"] = pd.to_datetime(
    electricity_prices["Date"],
    utc=True
)

electricity_prices = electricity_prices.set_index("Date").sort_index()

monthly_prices = electricity_prices["Price_DayAhead"].resample("ME").mean()
plt.figure(figsize=(10, 5))
plt.plot(monthly_prices, marker='o')
plt.title("Monthly Electricity Prices in Germany")
plt.ylabel("€/MWh")
plt.grid(True)
plt.show()
```



1.2 Data Parameters & Sources

1.2.1 ENTSO-E Transparency Platform

The primary data source for electricity market variables is the ENTSO-E Transparency Platform (transparency.entsoe.eu), the official publication platform of the European Network of Transmission System Operators for Electricity. Data was accessed via the `entsoe-py` Python library.

The day-ahead prices originate from the Single Day-Ahead Coupling (SDAC), a pan-European mechanism that couples national wholesale electricity markets. The market clearing occurs daily around 12:00 CET, determining prices for each delivery hour of the following day.

Column	Description	Unit	Source	Role
Date	Timestamp in hourly resolution (UTC)	-	-	Index
Price_DayAhead	Day-ahead electricity market clearing price	€/MWh	`query_day_ahead_prices`	Target (y)
Load_Forecast	Day-ahead forecasted total electricity demand	MW	`query_load_forecast`	Feature (X)

Column	Description	Unit	Source	Role
Solar	Day-ahead forecasted solar generation	MW	`query_wind_and_solar_forecast`	Feature (X)
Wind_Offshore	Day-ahead forecasted offshore wind generation	MW	`query_wind_and_solar_forecast`	Feature (X)
Wind_Onshore	Day-ahead forecasted onshore wind generation	MW	`query_wind_and_solar_forecast`	Feature (X)

1.2.2 Open-Meteo Weather API

To capture weather-driven effects on electricity demand and renewable generation, we sourced hourly weather forecast data from the Open-Meteo API (open-meteo.com), an open-source weather API. We used the Historical Forecast API (`historical-forecast-api.open-meteo.com`) for past data and the Forecast API (`api.open-meteo.com`) for current predictions. Weather data was aggregated from six major German cities: Hamburg, Berlin, Leipzig, Frankfurt, Cologne, and Munich.

Column	Description	Unit	API Parameter	Role
timestamp	Hourly timestamp (Europe/Berlin → UTC)	-	-	Index
Weather_Temp_Forecast	Mean air temperature at 2m across six cities	°C	`temperature_2m`	Feature (X)
Weather_Wind_Forecast	Mean wind speed at 100m across six cities	m/s	`windspeed_100m`	Feature (X)
Weather_Solar_Forecast	Mean global horizontal irradiance across six cities	W/m²	`shortwave_radiation`	Feature (X)

1.3 Data Cleaning

In [2]:
```
# setting starting point as January 1st, 2023
electricity_prices = electricity_prices[electricity_prices.index > "2023-01-
```

1.3.1 Unification

Because Europe's coupled day-ahead electricity market switched to 15-minute pricing for delivery starting 1 October 2025, ENTSO-E's published day-ahead series follows that market design. To avoid mixed-frequency time series (hourly, then quarter-hourly), the first step is to unify them by converting everything to hourly.

In [3]:
```
# compute time differences between consecutive timestamps
time_diffs = electricity_prices.index.to_series().diff()

hourly_step = pd.Timedelta(hours=1)

# find first timestamp where the step is smaller than 1 hour
change_point = time_diffs[time_diffs < hourly_step].index.min()

df_hourly = electricity_prices.resample("h").mean()
```

To make working with data easier, we are combinig the electricity and weather data in one data set. The goal is to achieve an hour-aligned merge. However, the timestamps in weather data are time-naive, while those in electricity data are time-aware. So, we decided to join the data sets using UTC in both of them.

In [4]:
```
weather = pd.read_csv(
    "weather_forecasts_aggregated.csv",
    index_col=0,
    parse_dates=True
)

# convert weather data to UTC
df_weather_hourly = (
    weather
    .tz_localize("Europe/Berlin", nonexistent="shift_forward", ambiguous=Fal
    .tz_convert("UTC")
)

# unify electricity & weather based on UTC
df = df_hourly.join(df_weather_hourly, how="inner")
df.head()
```

Out[4]:

	Price_DayAhead	Load_Forecast	Solar	Wind_Offshore	Wind_Onsho
2023-01-01 01:00:00+00:00	-1.47	39059.02	0.0	3403.22	35232.5
2023-01-01 02:00:00+00:00	-5.08	37631.41	0.0	3421.21	34975.(
2023-01-01 03:00:00+00:00	-4.49	37770.84	0.0	3420.97	34064.7
2023-01-01 04:00:00+00:00	-5.40	37342.54	0.0	3445.44	33617.5
2023-01-01 05:00:00+00:00	-5.02	35192.27	0.0	3462.39	33488.2

1.3.2 Cleaning

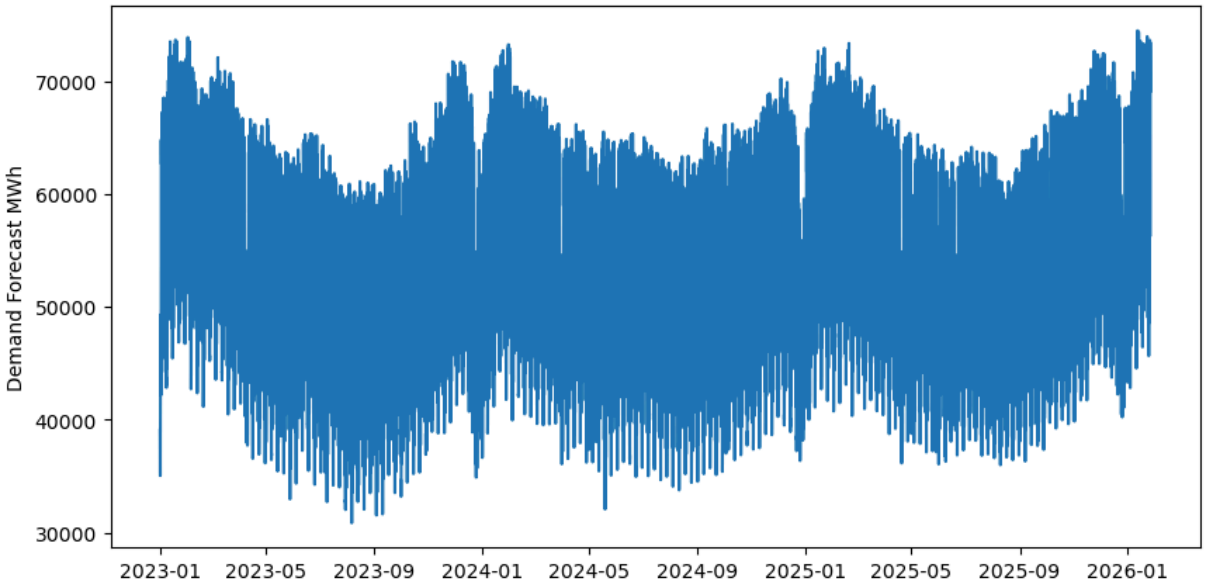
Next we will identify the missing values.

In [5]:
```
print(df.isna().sum())
```

```
Price_DayAhead            3
Load_Forecast             7
Solar                     8
Wind_Offshore             5
Wind_Onshore              8
Weather_Temp_Forecast     0
Weather_Wind_Forecast     0
Weather_Solar_Forecast    0
dtype: int64
```

Forecasted demand is one of the mising values. One of the best ways to handle missing values in time series is **time-series interpolation**. It is a method to estimate missing values using nearby known values. It assumes the variable changes smoothly over time and estimates missing points from neighboring timestamps, making it suitable for short gaps in continuous, ordered data. That being said, we firstly need to examine *Load_Forecast* to see whether it follows a particular trend.

In [6]:
```
plt.figure(figsize=(10, 5))
plt.plot(df.index, df["Load_Forecast"])
plt.ylabel("Demand Forecast MWh")
plt.show()
```



Conclusion: There is a strong seasonal trend. Fluctuations are smooth and continious. Therefore, for this case time-based interpolation is a good choice to handle missing values.

In [7]:
```
# apply time-based interpolation
df["Load_Forecast"] = (
    df["Load_Forecast"]
    .interpolate(method="time")
)
missing_load_forecast = df["Load_Forecast"].isna().sum()
if missing_load_forecast == 0:
    print("Load forecast has no more missing values.")
else:
    print("Load forecast has missing values.")
```

```
Load forecast has no more missing values.
```

The same approach applies to **solar** and **wind** production forecasts, since both are continuous physical processes. That being said, short gaps are best filled by **time interpolation** to preserve realistic, smooth temporal evolution rather than freezing (e.g. forward-fill) or flattening (mean-fill) the signal.

> Note: Zero values are actual legitimate values that represent no production (e.g. no solar energy production during the night).

In [8]:
```
for col in ["Solar", "Wind_Onshore", "Wind_Offshore"]:
    df[col] = (
        df[col]
        .interpolate(method="time", limit=3)
        .clip(lower=0)
    )

solar_wind_missing = df['Solar'].isna().sum() + df['Wind_Offshore'].isna().s

if solar_wind_missing == 0:
    print("Solar/wind forecast has no missing values.")
else:
    print("Solar/wind has missing values.")
```

```
Solar/wind forecast has no missing values.
```

There are also some missing **day ahead** prices. However, three missing prices in a multi-year hourly series are negligible. So, the best option, in our opinion, would be to just **drop** them.

In [9]:
```
df = df.dropna(subset=["Price_DayAhead"])
```

Final cleaning check:

In [10]:
```
print(df.isna().sum())
```

```
Price_DayAhead            0
Load_Forecast             0
Solar                     0
Wind_Offshore             0
Wind_Onshore              0
Weather_Temp_Forecast     0
Weather_Wind_Forecast     0
Weather_Solar_Forecast    0
dtype: int64
```

Part 2. Exploratory Data Analysis

2.1 Statistical summaries

In [11]:
```
df.describe()
```

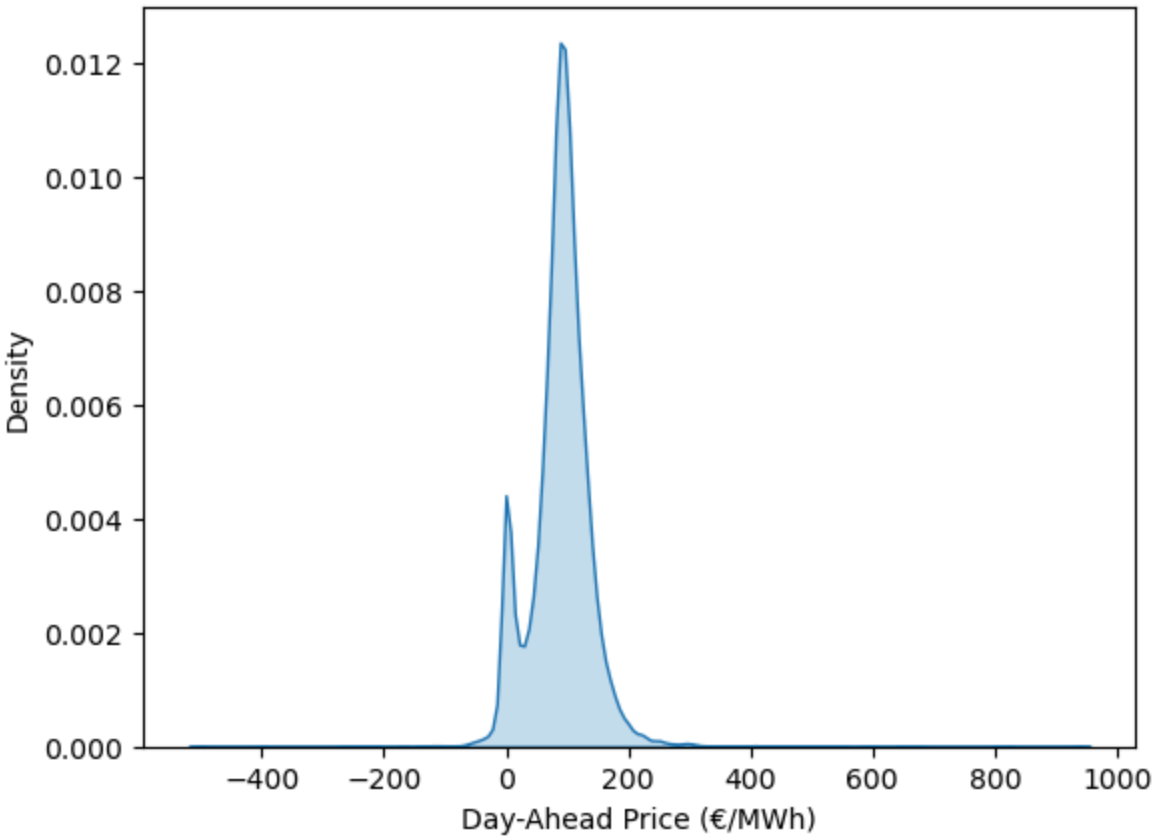
Out[11]:

	Price_DayAhead	Load_Forecast	Solar	Wind_Offshore	Wind_Onshore
count	26923.000000	26923.000000	26923.000000	26923.000000	26923.000000
mean	88.126220	53618.293782	7275.699765	2873.855240	12991.254028
std	51.161207	9057.178613	11164.463811	1829.996487	9993.710340
min	-500.000000	30848.600000	0.000000	12.300000	202.980000
25%	65.900000	46252.837500	0.000000	1190.797500	5074.695000
50%	90.540000	53377.570000	158.500000	2764.780000	10281.570000
75%	113.270000	61000.232757	11603.580000	4493.315000	18560.085000
max	936.280000	74511.838791	51062.840000	7097.032500	46793.530000

Day-Ahead Prices exhibit substantial variability but are not strongly skewed. The mean (≈ 88 €/MWh) and median (≈ 90 €/MWh) are very close, indicating a relatively balanced central tendency. The interquartile range spans roughly from 65 €/MWh (25th percentile) to 113 €/MWh (75th percentile), showing that half of the observed prices lie within a moderate range. At the same time, the full range extends from –500 €/MWh to over 900 €/MWh, revealing the presence of rare but extreme price events.

Below the distribution of day-ahead prices is demonstarted visually. It is dominated by a main peak at typical price levels but also shows a distinct concentration near zero. Its long tails reflect rare extreme price events.

In [12]:
```
import seaborn as sns

sns.kdeplot(df["Price_DayAhead"], fill=True)
plt.xlabel("Day-Ahead Price (€/MWh)")
plt.show()
```



Forecasted Demand Forecasted demand is much more stable. The mean and median are nearly identical, and the interquartile range is narrow compared to the total range, indicating a smooth and approximately symmetric distribution. The absence of extreme outliers reflects the predictable nature of electricity demand.

Solar production is strongly right-skewed and zero-inflated. The 25th percentile is zero, while the median is very small relative to the mean, showing that a large fraction of hours have little or no solar output (nighttime). **Wind offshore** and **Wind onshore** both show broad spreads. The mean exceeds the median in both cases, implying right-skewness, but unlike solar, the minimum values are strictly positive.

Weather temperature forecast is relatively symmetric, with mean and median close and a bounded range that aligns with realistic Central European temperatures. This suggests a well-behaved, near-normal distribution, which makes it easier for models to learn from. **Solar and wind weather forecasts** imply similar behaviour as solar and wind production forecasts, which is exactly what we would want to expect.

2.2 Investigation of Relationships

Demand <-> Price

Firstly, we will look at how the forecasted demand correlates with actual day-ahead electricity prices.

In [13]:
```
from scipy import stats
spearman_corr, _ = stats.spearmanr(df['Load_Forecast'], df['Price_DayAhead']
pearson_corr = df['Load_Forecast'].corr(df['Price_DayAhead'])
print(f"Pearson rank corr: {pearson_corr}")
print(f"Spearman rank corr: {spearman_corr}")
```

```
Pearson rank corr: 0.2899338214559749
Spearman rank corr: 0.281881432078017
```

Interpretation: higher expected demand tends to coincide with higher prices, but demand alone explains only a limited part of price variation (weak correlation). Electricity prices correlate more with residual load, since demand on isolation does not imply much information (e.g. high demand with high renewables would not result in higher prices; moderate demand with low renewables could still result in high prices). Additionally, fuel prices, imports, bidding behavior add noise.

(Demand & Renewables) <-> Price

Now, we will look at how the forecasted demand correlates with forecasted energy production from renewables using the formula for residual load. Residual load is the part of electricity demand that remains after subtracting variable renewable generation. The formula for hour *t* is

$$\text{Residual Load}_t = \text{Load}_t - \text{Wind}_t - \text{Solar}_t$$

High residual load means that renewables cower little of demand. Therefore, more conventional plants are needed, which results in higher electricity prices. It works the opposite way for low residual load, since in this case renewables cower most/all demand.

This implies that residual load should strongly correlate with electricity prices. So, this is what we'll be testing next using **Spearman's rank** correlation. This correlation type is better suited for electricity price analysis because it captures monotonic but nonlinear relationships and is robust to extreme price spikes (e.g. sharp price spikes during scarcity events).

In [14]:
```
from scipy import stats
residual_load = df['Load_Forecast'] - df['Solar'] - df['Wind_Offshore'] - df
corr, _ = stats.spearmanr(df['Price_DayAhead'], residual_load)
print(f"Correlation between electricity prices and residual load: {corr}")
```

```
Correlation between electricity prices and residual load: 0.860202146485559
```

Interpretation: The residual load shows a strong positive correlation with day-ahead prices. This indicates that higher residual load leads to higher day-ahead electricity. Even though substantial variability still remains due to unexplained market, fuel, and

network effects, correlation rate of 0.86 motivates adding Residual Load as a model parameter.

Demand <-> Weather

Next we will look at how forecasted weather correlates with forecasted demand. In this case we would expect the correlation index to be very low. This is because the relationship between these two variales is non-linear. Intuitively it is expected that cold weather leads to higher electricity prices (heating demand), as well as hot weather (cooling demand), while mild weather results in lower prices. So, it is better to investigate this relationship separately for hot and cold weather.

In [15]:
```
print(f"Demand to weather correlation: {df['Load_Forecast'].corr(df['Weather
cold_temp = df.query('Weather_Temp_Forecast <= 0')['Weather_Temp_Forecast']
print(f"Demand to COLD weather correlation: {df['Load_Forecast'].corr(cold_t
hot_temp = df.query('Weather_Temp_Forecast >= 27')['Weather_Temp_Forecast']
print(f"Demand to HOT weather correlation: {df['Load_Forecast'].corr(hot_tem
```

```
Demand to weather correlation: -0.15155616493153048
Demand to COLD weather correlation: -0.01674965969138792
Demand to HOT weather correlation: -0.019976108314476947
```

Interpretation: Even after calculating the correlation between two weather conditions separately, actual average temperature seem to significantly affect neither the expected demand or, therefore, the price.

Renewables <-> Weather

Since residual load has a moderate to strong correlation with day-ahead prices, it is good to investigate the fact whether forecasted weather significantly impacts forecasted renewable energy production.

In [16]:
```
print(f"Solar production to shortwave radiation corr: {df['Solar'].corr(df['

df["Wind_Mean"] = df[["Wind_Offshore", "Wind_Onshore"]].mean(axis=1)
corr = df["Wind_Mean"].corr(df["Weather_Wind_Forecast"])
print(f"Mean wind production to wind speed corr: {corr}")
```

```
Solar production to shortwave radiation corr: 0.9706075601418002
Mean wind production to wind speed corr: 0.8910607125466447
```

Interpretation: Since the correlation is very strong in both cases, weather features, which we gathered, are meaningful and not random noise.

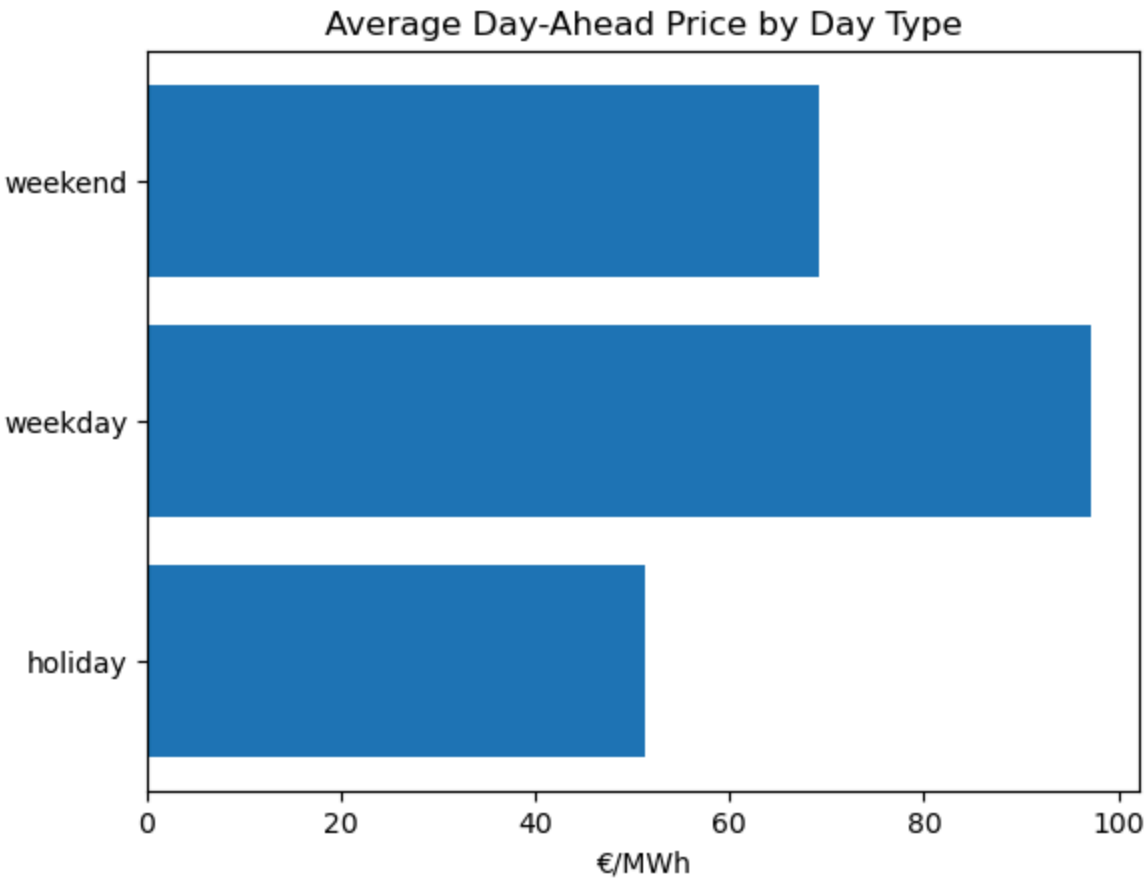
Day Type <-> Price

Because electricity demand and market behavior differ systematically between weekdays, weekends, and public holidays, it is also important to investigate the

relationship between day types and electricity prices. Weekdays typically exhibit higher industrial and commercial demand, leading to higher prices, while weekends and holidays show reduced activity and lower demand, often resulting in lower prices. These calendar effects are highly regular and cannot be fully captured by lagged prices or weather variables alone.

In [17]:
```
!pip install holidays
import matplotlib.pyplot as plt
import holidays

# extract holidays for Germany for time period in our data set
years = range(df.index.year.min(), df.index.year.max() + 1)
de_holidays = holidays.Germany(years=years)

# assign day types
df['is_holiday'] = pd.Series(df.index.date, index=df.index).isin(de_holidays
df['is_weekend'] = df.index.weekday >= 5
df['is_weekday'] = ~df['is_weekend']
df['day_type'] = 'weekday'
df.loc[df['is_weekend'], 'day_type'] = 'weekend'
df.loc[df['is_holiday'], 'day_type'] = 'holiday'

day_type_prices_avg = df.groupby('day_type')['Price_DayAhead'].mean().reset_

plt.figure()
plt.barh(day_type_prices_avg['day_type'], day_type_prices_avg['Price_DayAhea
plt.xlabel('€/MWh')
plt.title('Average Day-Ahead Price by Day Type')
plt.show()
```

```
Requirement already satisfied: holidays in /opt/anaconda3/lib/python3.13/sit
e-packages (0.87)
Requirement already satisfied: python-dateutil<3,>=2.9.0.post0 in /opt/anaco
nda3/lib/python3.13/site-packages (from holidays) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.13/sit
e-packages (from python-dateutil<3,>=2.9.0.post0->holidays) (1.17.0)
```



Interpretation: the above mentioned expectations are confirmed.

- Weekdays: higher industrial and commercial demand -> higher prices
- Weekends: reduced economic activity -> lower demand -> lower prices
- Public holidays: demand often drops even further than on weekends

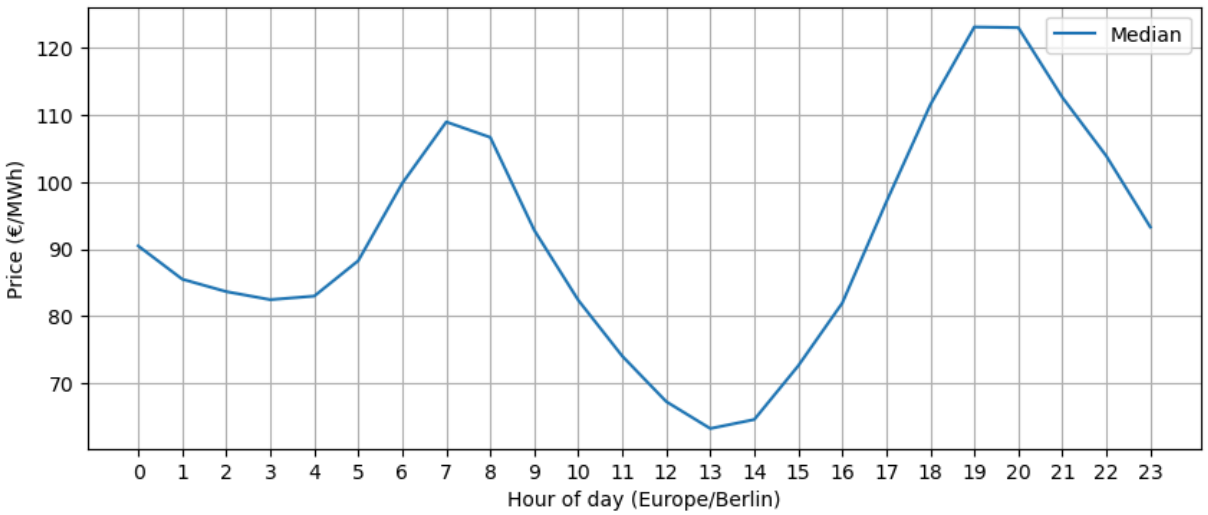
This motivates adding day type as a model parameter.

Day Time <-> Price

Next we will be investigating the relationship between day-ahead electricity prices and the hour of the day. This is essential, because intuitively market bidding behavior should strongly follow recurring intraday cycles. To do so, we are plotting the median of prices for every hour of they day. The median is chosen, because according to the statistical analysis done earlier, the day-ahead prices have some extreme outliers and median is more stable for cases like this one.

In [18]:
```
df_berlin = df.copy()
df_berlin['hour'] = df_berlin.index.tz_convert("Europe/Berlin").hour
hour_profile = df_berlin.groupby('hour')['Price_DayAhead'].agg(['median'])

plt.figure(figsize=(10, 4))
plt.plot(hour_profile.index, hour_profile["median"], label="Median")
plt.xticks(range(0, 24))
```

```
plt.xlabel("Hour of day (Europe/Berlin)")
plt.ylabel("Price (€/MWh)")
plt.grid(True)
plt.legend()
plt.show()
```



Interpretation:

- Night hours (01–05): lower prices due to reduced demand
- Morning peak (07–08): rising prices as demand increases
- Midday dip (12–14): lower prices (probably partly driven by solar generation)
- Evening peak (18–21): highest prices, reflecting peak demand and limited supply flexibility

Such seasonality is a strong motivator to use day time is model parameter. Capturing those patterns could allow models to distinguish structural daily price effects from random market fluctuations.

Part 3. Modelling

Level 0: Naive persistance (baseline)

As a reference point we use a naive persistence model as a baseline. The baseline assumes that the day-ahead electricity price for a given delivery hour is equal to the realized price of the same hour on the previous day. Formally, the forecast is defined as

$$\hat{P}_t = P_{t-24}$$

Model performance is evaluated using the **Mean Absolute Error (MAE)** and the **Root Mean Squared Error (RMSE)**. MAE describes the typical difference between predicted and observed prices. RMSE gives more weight to larger errors. Using both measures helps to assess average prediction accuracy and the effect of occasional larger

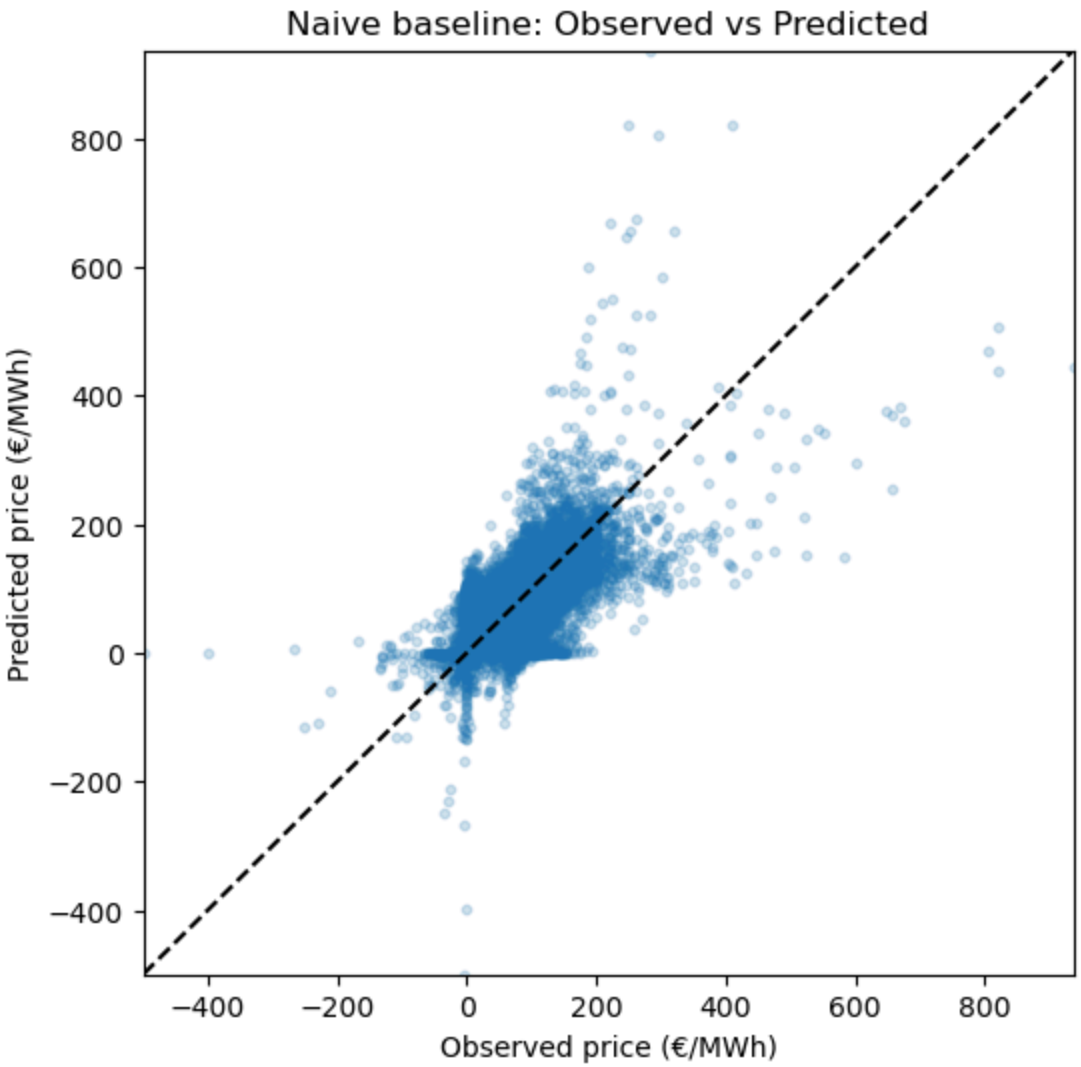
deviations. We will be using the values calculated for the baseline model to evaluate the performence of the following models.

In [19]:
```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error

df_base = df.copy()

# Target: Price 24 hours in the future (next day)
df_base["y_true"] = df_base["Price_DayAhead"].shift(-24)
# Prediction: Use today's price as forecast for tomorrow
df_base["y_pred_lvl0"] = df_base["Price_DayAhead"]

# Drop rows where target is NaN (last 24 hours)
df_base = df_base.dropna(subset=["y_true", "y_pred_lvl0"])

y_test = df_base["y_true"]
p_test = df_base["y_pred_lvl0"]

# computing metrics
mae_base = mean_absolute_error(y_test, p_test)
rmse_base = mean_squared_error(y_test, p_test) ** 0.5

print(f"MAE : {mae_base:.3f} €/MWh")
print(f"RMSE: {rmse_base:.3f} €/MWh")
```

```
MAE : 27.023 €/MWh
RMSE: 41.772 €/MWh
```

The figure below shows the observed versus predicted day-ahead electricity prices for the naive persistence baseline, where the forecast for each hour equals the observed price 24 hours earlier.

In [20]:
```
plt.figure(figsize=(6, 6))
plt.scatter(y_test, p_test, alpha=0.2, s=10)
lims = [
    min(y_test.min(), p_test.min()),
    max(y_test.max(), p_test.max())
]

plt.plot(lims, lims, linestyle="--", color="black")
plt.xlim(lims)
plt.ylim(lims)
plt.xlabel("Observed price (€/MWh)")
plt.ylabel("Predicted price (€/MWh)")
plt.title("Naive baseline: Observed vs Predicted")
plt.show()
```



Plot summary: Most observations lie close to the diagonal, indicating that the baseline captures short-term price persistence. However, at moderately high price levels around 200 €/MWh, predictions begin to deviate noticeably from the diagonal. This indicates that the naive baseline loses accuracy already outside typical price ranges and is unable to reliably anticipate larger price movements.

Level 1: SARIMA

The Seasonal ARIMA (SARIMA) model is a univariate time-series forecasting method that extends the classical ARIMA framework by explicitly accounting for seasonal patterns. It models the current value of a time series as a function of its past values, past forecast errors, and recurring seasonal effects.

SARIMA relies solely on the historical values of the target variable and does not incorporate external explanatory information. As a result it cannot directly respond to changes in the fundamental drivers, which will be discuseed later in the project. SARIMA

is therefore used as our intermediate model that remains less expressive than models with exogenous variables.

Step 1: Preparing data and extracting target & parameters

A fixed hourly frequency is required by SARIMAX to correctly model temporal dependencies and seasonal patterns, especially the strong 24-hour cycle present in electricity prices. Thus this is done by explicitly setting the Pandas time frequency to hourly.

In [21]:
```
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller

pd.set_option('future.no_silent_downcasting', True)
df_sar = df.copy()
df_sar = df_sar[~df_sar.index.duplicated(keep="first")]
df_sar = df_sar.asfreq("h")
df_sar = df_sar.ffill().bfill()

df_sar = df_sar.dropna()

# target
y = df_sar['Price_DayAhead']
```

Step 2: Dividing data into train, validate and test sets

The data are divided into training, validation, and test sets using a chronological split. Everything up to June 1st, 2025 is used for training, while 20% of it is reserved for validation for hyperparamater tuning. The rest of the data is used for testing.

In [22]:
```
split_time_test = "2025-06-01"
y_train_full = y.loc[y.index < split_time_test]
y_test = y.loc[y.index >= split_time_test]

# validation (last 20% of training)
val_frac = 0.2
cut = int(len(y_train_full) * (1 - val_frac))
y_train = y_train_full.iloc[:cut]
y_val   = y_train_full.iloc[cut:]
print(f"Training samples: {len(y_train)}")
print(f"Validation samples: {len(y_val)}")
print(f"Test samples: {len(y_test)}")
```

```
Training samples: 16933
Validation samples: 4234
Test samples: 5759
```

Step 3: Hyperparameter tuning with cross-validation

A Seasonal ARIMA (SARIMA) model is specified as

$$\text{SARIMA}(p,d,q)(P,D,Q,s)$$

where

Parameter	Description
`p`	How many past values the model looks at
`d`	How many times the series is differenced to remove trends
`q`	How many past errors are used
`P`	How many past seasonal values the model looks at
`D`	How many times seasonal patterns are removed
`Q`	How many past seasonal errors are used
`s`	Length of one season (e.g. 24 hours for daily seasonality)

Code below is used to identify the best hyperparameter configuration, a grid search is performed over a restricted set of SARIMA orders with p,q∈{0,1,2}, d∈{0,1}, P,D,Q∈{0,1}. The seasonal period is set to s=24 to reflect daily seasonality in hourly electricity prices. For each candidate configuration the model is estimated using the training data and its performance is evaluated on a separate validation set. The configuration that has the lowest validation error is selected as the final model. The code is commented out due to runtime considerations, while the resulting parameter choices are used in the final SARIMA specification.

In [23]:
```
import itertools

def fit_sarima(y_train, order, seasonal_order):
    model = SARIMAX(
        y_train,
        order=order,
        seasonal_order=seasonal_order,
        enforce_stationarity=False,
        enforce_invertibility=False
    )
    return model.fit(disp=False)

def eval_forecast(res, y_val):
    fc = res.get_forecast(steps=len(y_val)).predicted_mean
    mae = mean_absolute_error(y_val, fc)
    rmse = mean_squared_error(y_val, fc) ** 0.5
    return mae, rmse

# FINDING BEST HYPERPARAMETERS
# p = [0, 1, 2]
# d = [0, 1]
# q = [0, 1, 2]

# P = [0, 1]
# D = [0, 1]
```

```
# Q = [0, 1]

# s = 24  # daily seasonality for hourly data

# orders = list(itertools.product(p, d, q))
# seasonals = [(P_, D_, Q_, s) for P_ in P for D_ in D for Q_ in Q]

# results_list = []

# step=0
# for order in orders:
#     for seas in seasonals:
#         try:
#             res = fit_sarima(y_train, order, seas)
#             mae, rmse = eval_forecast(res, y_val)
#             results_list.append({
#                 "order": order,
#                 "seasonal_order": seas,
#                 "val_mae": mae,
#                 "val_rmse": rmse,
#                 "aic": res.aic
#             })
#             print(step)
#             step = step + 1
#         except Exception:
#             # skip if combo fails to converge
#             continue

# results_df = pd.DataFrame(results_list).sort_values("val_mae").reset_index
# best = results_df.iloc[0]
# best_order = tuple(best["order"])
# best_seasonal = tuple(best["seasonal_order"])

# print("\nBest by validation MAE:")
# print("order =", best_order)
# print("seasonal_order =", best_seasonal)
```

Step 4: Predicting & Evaluating

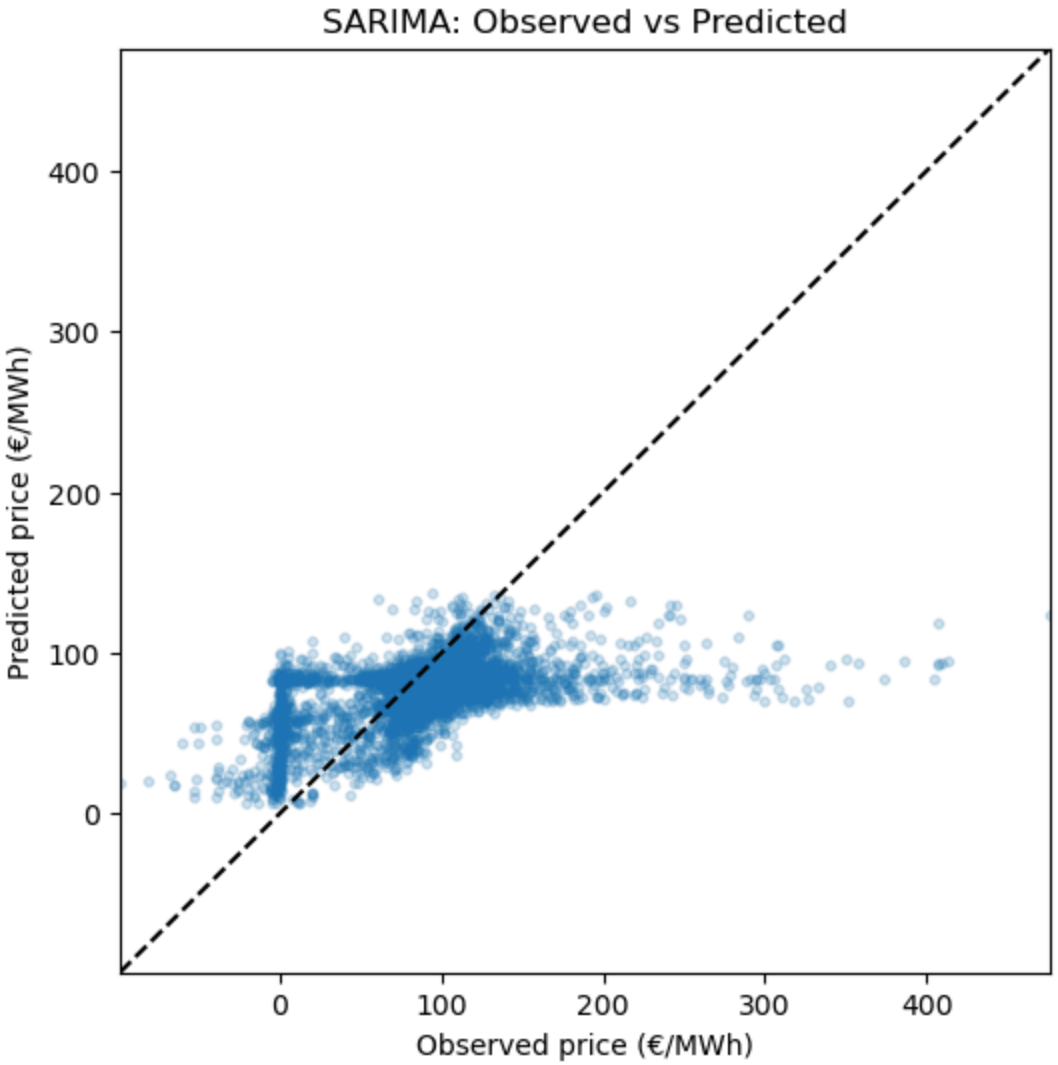
In [24]:
```
# fitting the model using the best params combo from prev step
best_res = fit_sarima(y_train_full, (2, 1, 2), (1, 0, 1, 24))
test_fc = best_res.get_forecast(steps=len(y_test)).predicted_mean

mae_sar = mean_absolute_error(y_test, test_fc)
rmse_sar = mean_squared_error(y_test, test_fc) ** 0.5

print(f"MAE : {mae_sar:.3f} €/MWh")
print(f"RMSE: {rmse_sar:.3f} €/MWh")
```

```
MAE : 28.446 €/MWh
RMSE: 42.488 €/MWh
```

Error interpretation

Both MAE and RMSE are nearly identical for SARIMA and the baseline model. This indicates that SARIMA does not provide a meaningful improvement over a naive persistence forecast and fails to extract additional predictive structure from the data.

In [25]:
```
plt.figure(figsize=(6, 6))
plt.scatter(y_test, test_fc, alpha=0.2, s=10)
lims = [
    min(y_test.min(), test_fc.min()),
    max(y_test.max(), test_fc.max())
]
plt.plot(lims, lims, linestyle="--", color="black")
plt.xlim(lims)
plt.ylim(lims)
plt.xlabel("Observed price (€/MWh)")
plt.ylabel("Predicted price (€/MWh)")
plt.title("SARIMA: Observed vs Predicted")
plt.show()
```



Plot summary: the plot shows a systematic flattening of predictions at higher price levels, where points increasingly fall below the diagonal. As a result high price spikes are systematically underpredicted. This behavior leads to frequent moderate errors

(reflected in a slightly higher MAE). At the same time extremely large deviations are avoided, which keeps the RMSE at a level comparable to the naive baseline.

Level 2: Linear Regression

Our linear regression model use a compact set of explanatory variables that represent the dominant structural drivers of day-ahead electricity prices.

- The main variable is **forecasted residual load**. Residual load reflects the amount of demand that must be covered by conventional generation. EDA showed a substantially stronger relationship of residual load with the prices than with demand alone. Solar and wind forecasts are not included separately, since their effect is already contained in residual load. Therefore, doing so could introduce strong multicollinearity.
- To capture short-term price persistence, the **day-ahead price lagged by 24 hours** is added. This variable represents systematic bidding behavior and daily market structure.
- Calendar effects are modeled using binary indicators for **weekends** and **public holidays**.
- Intraday seasonality is represented by **sine and cosine transformations of the hour of day**. Hours are encoded trigonometrically, since hour 23 and hour 0 are adjacent in reality, but numerically they are far apart, which distorts any linear relationship. Sine and cosine map the 24-hour clock onto a circle to preserve the cyclical nature of time.

> **Weather forecast variables** are intentionally excluded from our linear regression model, because they exhibit very high correlations with solar and wind forecasts. The last ones are already included through residual load. So, adding weather data would primarily add redundant information and could degrade stability of the model. However, we will be still investigating whether they can help achieve better predictions in our non-linear model - XGBoost.

Step 1: Extracting parameters and target

In [26]:
```
df_lr = df.copy()

# target
df_lr["Price_Target"] = df_lr["Price_DayAhead"].shift(-24)

df_lr['Residual_Load'] = df_lr['Load_Forecast'] - df_lr['Solar'] - df_lr['Wi
df_lr['Price_lag_24'] = df_lr['Price_DayAhead']
```

```
df_lr["Residual_Load_shift_24"] = df_lr["Residual_Load"].shift(-24)

# intraday seasonality
target_index = df_lr.index + pd.Timedelta(hours=24)
target_hour = target_index.tz_convert("Europe/Berlin").hour
df_lr["hour_sin"] = np.sin(2 * np.pi * target_hour / 24.0)
df_lr["hour_cos"] = np.cos(2 * np.pi * target_hour / 24.0)


# day-type flags
df_lr["is_weekend"] = (target_index.weekday >= 5).astype(int)
df_lr["is_holiday"] = pd.Series(target_index.date, index=df_lr.index).isin(c

# feature matrix
X = df_lr[["Residual_Load_shift_24", "Price_lag_24", "is_weekend", "is_holic

# dropping rows with NaNs created by lagging (first 24 hours)
y = df_lr["Price_Target"]
data = pd.concat([y, X], axis=1).dropna()
y = data["Price_Target"]
X = data.drop(columns=["Price_Target"])
```

Step 2: Diving data into train and test sets

Since the linear regression model does not require hyperparameter tuning, a simple train–test split with 80/20 proportion should be sufficient.

In [27]:
```
split_time = "2025-06-01" # starting point for testing
train_mask = X.index < split_time
test_mask  = ~train_mask
X_train, y_train = X.loc[train_mask], y.loc[train_mask]
X_test,  y_test  = X.loc[test_mask],  y.loc[test_mask]

print(f"Training samples: {len(X_train)}")
print(f"Test samples: {len(X_test)}")
```

```
Training samples: 21166
Test samples: 5733
```

Step 3: Standardizing data

Prior to model estimation, all explanatory variables are standardized using z-score normalization. This is necessary because the predictors operate on very different numerical ranges. For example, residual load typically varies between 10 000–60 000 MW, lagged prices between roughly 0–200 €/MWh, while time-based features (sin, cos) are bounded within [–1, 1]. Standardization prevents scale differences from dominating the estimation.

In [28]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

X_train_scaled = pd.DataFrame(
```

```
    scaler.fit_transform(X_train),
    index=X_train.index,
    columns=X_train.columns
)

X_test_scaled = pd.DataFrame(
    scaler.transform(X_test),
    index=X_test.index,
    columns=X_test.columns
)
```

Step 4: Predicting & Evaluating

In [29]:

```
import statsmodels.api as sm

lr_model = sm.OLS(y_train, sm.add_constant(X_train_scaled)).fit()
y_pred = lr_model.predict(sm.add_constant(X_test_scaled))

print(f"SUMMARY: \n {lr_model.summary()}")
mae_lr = mean_absolute_error(y_test, y_pred)
rmse_lr = mean_squared_error(y_test, y_pred) ** 0.5
print("-------------------------------------")
print(f"MAE : {mae_lr:.3f} €/MWh")
print(f"RMSE: {rmse_lr:.3f} €/MWh")
```

SUMMARY:

OLS Regression Results

Dep. Variable:	Price_Target	R-squared:	0.742
Model:	OLS	Adj. R-squared:	0.742
Method:	Least Squares	F-statistic:	1.013e+04
Date:	Sun, 25 Jan 2026	Prob (F-statistic):	0.00
Time:	20:23:16	Log-Likelihood:	-99280.
No. Observations:	21166	AIC:	1.986e+05
Df Residuals:	21159	BIC:	1.986e+05
Df Model:	6		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	88.4356	0.181	488.130	0.000	88.080	88.791
Residual_Load_shift_24	33.7825	0.238	141.923	0.000	33.316	34.249
Price_lag_24	14.7878	0.225	65.766	0.000	14.347	15.229
is_weekend	-1.2287	0.195	-6.305	0.000	-1.611	-0.847
is_holiday	-1.1878	0.184	-6.438	0.000	-1.549	-0.826
hour_sin	-0.8534	0.182	-4.678	0.000	-1.211	-0.496
hour_cos	0.8188	0.183	4.473	0.000	0.460	1.178

Omnibus:	20545.937	Durbin-Watson:	0.273
Prob(Omnibus):	0.000	Jarque-Bera (JB):	5857817.334
Skew:	4.101	Prob(JB):	0.00
Kurtosis:	84.086	Cond. No.	2.19

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
-----------------------------------
MAE : 16.247 €/MWh
RMSE: 23.982 €/MWh
```

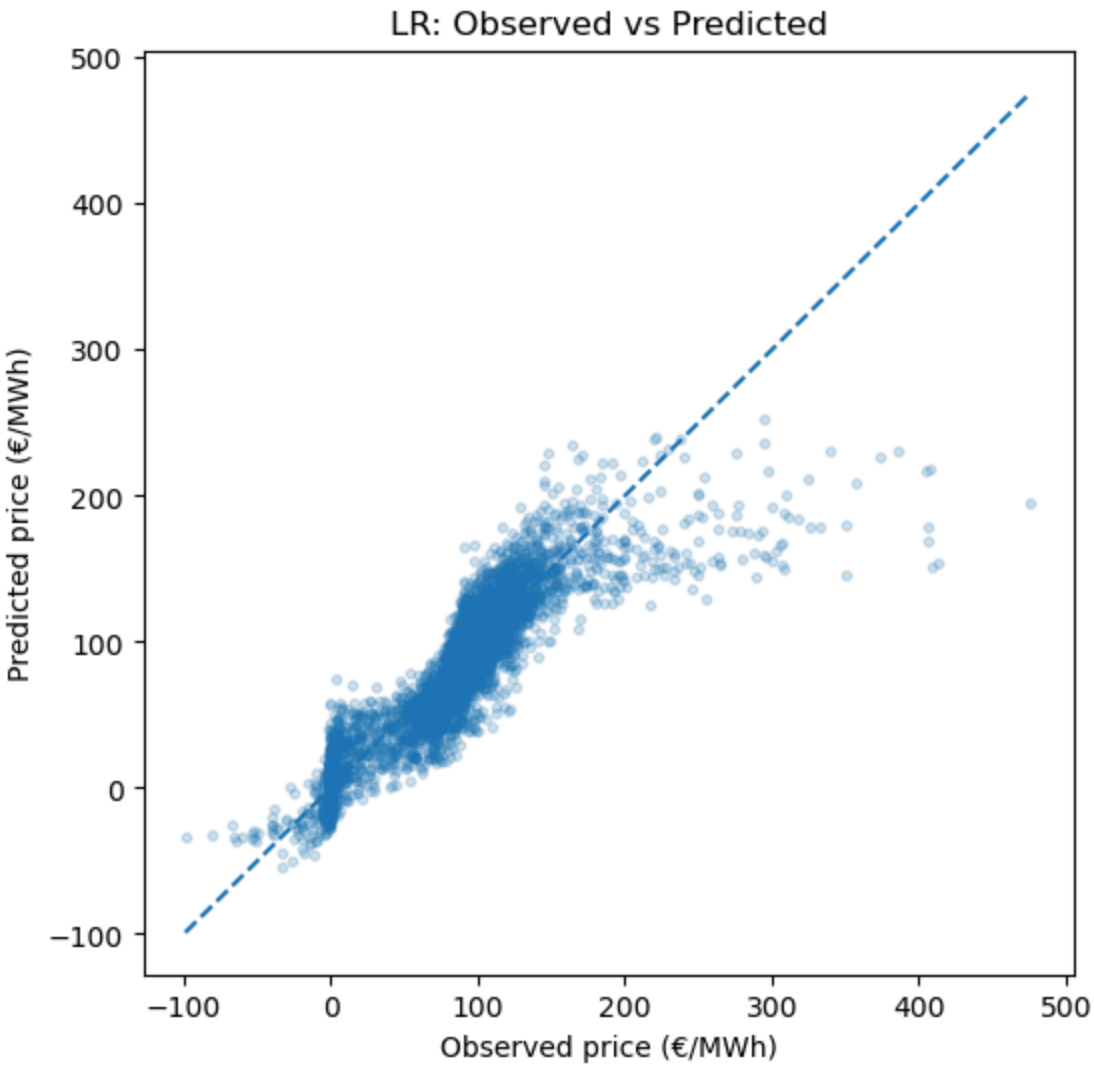
Interpretation of the summary

- High explanatory power ($R^2 \approx 0.70$): around 70% of the variance in day-ahead electricity prices is explained by the included predictors.
- Jointly statistically significant (F-statistic p-value = 0). This confirms that our model parameters provide meaningful information beyond a constant baseline.
- Residual load is the strongest predictor. It has a positive coefficient, confirming that higher residual load is associated with higher electricity prices.

Error interpretation

Compared to the baseline persistence model (MAE ≈ 28 €/MWh, RMSE ≈ 42 €/MWh), the linear regression model substantially improves predictive performance, reducing the MAE to ~16 €/MWh and the RMSE to around 23 €/MWh. This indicates both better average accuracy and fewer large forecast errors. To understand model's performance even better, observed and predicted prices are depicted in the plot below.

In [30]:
```
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred, alpha=0.2, s=10)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         linestyle="--")
plt.title("LR: Observed vs Predicted")
plt.xlabel("Observed price (€/MWh)")
plt.ylabel("Predicted price (€/MWh)")
plt.show()
```



Plot summary: The most of observations lies close to the diagonal line for low to moderate prices, indicating that the model predicts typical market conditions reasonably well. Compared to SARIMA and the naive baseline, linear regression achieves notably lower MAE and RMSE. However, the plot still shows that extreme price spikes remain underestimated. This highlights the limitations of a purely linear model.

Level 3: XGBoost

XGBoost (eXtreme Gradient Boosting) is a tree-based ensemble method that builds models sequentially, where each new tree corrects the errors of the previous ones. Unlike linear regression, XGBoost can capture nonlinear relationships and feature interactions without explicit specification.

Step 1: Extracting features and target

The model uses the same predictors as linear regression:

- Forecasted residual load as the primary driver of conventional generation dispatch

- Day-ahead price lagged by 24 hours to capture price persistence
- Weekend and holiday indicators for the target day
- Sine and cosine transformations of the hour for intraday seasonality

Additionally, we test whether adding weather forecast variables (temperature, wind speed, solar radiation) improves prediction accuracy. While these were excluded from linear regression due to high correlation with the ENTSO-E generation forecasts (which would cause multicollinearity), XGBoost as a tree-based method is robust to correlated features. We compare both variants to quantify the added value of weather data.

In [31]:
```
df_xgb = df.copy()

# Target: Price 24 hours in the future
df_xgb["Price_Target"] = df_xgb["Price_DayAhead"].shift(-24)

# Features from ENTSO-E
df_xgb["Residual_Load"] = (
    df_xgb["Load_Forecast"] - df_xgb["Solar"] -
    df_xgb["Wind_Onshore"] - df_xgb["Wind_Offshore"]
)
df_xgb["Residual_Load_shift_24"] = df_xgb["Residual_Load"].shift(-24)

# Price lag:
df_xgb["Price_lag_24"] = df_xgb["Price_DayAhead"]

# Calendar features for TARGET day
target_index = df_xgb.index + pd.Timedelta(hours=24)
target_hour = target_index.tz_convert("Europe/Berlin").hour
df_xgb["hour_sin"] = np.sin(2 * np.pi * target_hour / 24.0)
df_xgb["hour_cos"] = np.cos(2 * np.pi * target_hour / 24.0)

# Weekend/holiday flags for target day
df_xgb["is_weekend"] = (target_index.weekday >= 5).astype(int)
target_dates = pd.Series(target_index.date, index=df_xgb.index)
df_xgb["is_holiday"] = target_dates.isin(de_holidays).astype(int)

# Feature matrix WITH weather variables
features = [
    "Residual_Load_shift_24", "Price_lag_24", "is_weekend", "is_holiday",
    "hour_sin", "hour_cos",
    "Weather_Temp_Forecast", "Weather_Wind_Forecast", "Weather_Solar_Forecas
]

# Feature matrix WITHOUT weather (for comparison)
features_no_weather = [
    "Residual_Load_shift_24", "Price_lag_24", "is_weekend", "is_holiday",
    "hour_sin", "hour_cos"
]

X = df_xgb[features]
y = df_xgb["Price_Target"]

# Drop rows with NaN (last 24 hours have no target)
```

```
data = pd.concat([y, X], axis=1).dropna()
y = data["Price_Target"]
X = data.drop(columns=["Price_Target"])

print(f"Feature matrix shape: {X.shape}")
print(f"Features: {list(X.columns)}")
print(f"Date range: {X.index.min()} to {X.index.max()}")
```

```
Feature matrix shape: (26899, 9)
Features: ['Residual_Load_shift_24', 'Price_lag_24', 'is_weekend', 'is_holid
ay', 'hour_sin', 'hour_cos', 'Weather_Temp_Forecast', 'Weather_Wind_Forecas
t', 'Weather_Solar_Forecast']
Date range: 2023-01-01 01:00:00+00:00 to 2026-01-25 22:00:00+00:00
```

Step 2: Dividing data into train and test sets

The same temporal split as in linear regression is used to ensure comparability. All data before 2025-06-01 is used for training (including hyperparameter tuning), and all data onward is held out for final evaluation.

In [32]:
```
split_time = "2025-06-01"
train_mask = X.index < split_time
test_mask = ~train_mask

X_train, y_train = X.loc[train_mask], y.loc[train_mask]
X_test, y_test = X.loc[test_mask], y.loc[test_mask]

print(f"Training samples: {len(X_train)}")
print(f"Test samples: {len(X_test)}")
```

```
Training samples: 21166
Test samples: 5733
```

Step 3: Hyperparameter tuning with time-series cross-validation

XGBoost has several hyperparameters that control model complexity. The most important ones are:

Parameter	Description
`n_estimators`	Number of boosting rounds (trees)
`max_depth`	Maximum depth of each tree
`learning_rate`	Step size shrinkage to prevent overfitting
`min_child_weight`	Minimum sum of instance weight needed in a child node
`subsample`	Fraction of training samples used per tree

To find optimal values, we use TimeSeriesSplit cross-validation, which creates expanding training windows while always validating on future data. This respects the temporal structure of the data and prevents information leakage from future to past.

In [33]:
```
!pip install xgboost
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
import xgboost as xgb

# define parameter grid
param_grid = {
    "n_estimators": [100, 500, 1000],
    "max_depth": [2, 3, 4],
    "learning_rate": [0.01, 0.05],
    "min_child_weight": [10, 15, 20],
    "subsample": [0.6, 0.7, 0.8]
}

# time-series cross-validation with 5 splits
tscv = TimeSeriesSplit(n_splits=5)

# base model
xgb_model = xgb.XGBRegressor(
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)

# grid search
grid_search = GridSearchCV(
    estimator=xgb_model,
    param_grid=param_grid,
    cv=tscv,
    scoring="neg_mean_absolute_error",
    verbose=1,
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print(f"\nBest parameters: {grid_search.best_params_}")
print(f"Best CV MAE: {-grid_search.best_score_:.3f} €/MWh")
```

```
Requirement already satisfied: xgboost in /opt/anaconda3/lib/python3.13/site
-packages (3.1.3)
Requirement already satisfied: numpy in /opt/anaconda3/lib/python3.13/site-p
ackages (from xgboost) (2.1.3)
Requirement already satisfied: scipy in /opt/anaconda3/lib/python3.13/site-p
ackages (from xgboost) (1.15.3)
Fitting 5 folds for each of 162 candidates, totalling 810 fits

Best parameters: {'learning_rate': 0.05, 'max_depth': 2, 'min_child_weight':
20, 'n_estimators': 100, 'subsample': 0.7}
Best CV MAE: 17.937 €/MWh
```

Step 4: Training final model and evaluation

The final model is trained on the full training set using the optimal hyperparameters found during cross-validation. Unlike linear regression, XGBoost does not require feature

standardization.

In [34]:
```
best_model = grid_search.best_estimator_
y_pred_xgb = best_model.predict(X_test)

# evaluation metrics
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))

print("XGBoost Performance:")
print(f"MAE:  {mae_xgb:.3f} €/MWh")
print(f"RMSE: {rmse_xgb:.3f} €/MWh")
```

```
XGBoost Performance:
MAE:  14.058 €/MWh
RMSE: 23.389 €/MWh
```

Comparison: With vs. without weather variables

To evaluate whether the weather variables contribute meaningful information, we train a second XGBoost model using only the core features (without weather). To ensure a fair comparison, we perform a separate hyperparameter search for the reduced feature set.

In [35]:
```
# Comparison: Model WITHOUT weather variables
X_nw = df_xgb[features_no_weather]
y_nw = df_xgb["Price_Target"]

# Drop rows with NaN
data_nw = pd.concat([y_nw, X_nw], axis=1).dropna()
y_nw = data_nw["Price_Target"]
X_nw = data_nw.drop(columns=["Price_Target"])

# Same train/test split
X_train_nw, y_train_nw = X_nw.loc[train_mask], y_nw.loc[train_mask]
X_test_nw, y_test_nw = X_nw.loc[test_mask], y_nw.loc[test_mask]

# Separate grid search for model without weather
grid_search_nw = GridSearchCV(
    estimator=xgb.XGBRegressor(objective="reg:squarederror", random_state=42
    param_grid=param_grid,
    cv=tscv,
    scoring="neg_mean_absolute_error",
    verbose=1,
    n_jobs=-1
)
grid_search_nw.fit(X_train_nw, y_train_nw)

print(f"\nBest parameters (no weather): {grid_search_nw.best_params_}")

# Train final model and predict
model_no_weather = grid_search_nw.best_estimator_
y_pred_nw = model_no_weather.predict(X_test_nw)

# Comparison
```

```
mae_no_weather = mean_absolute_error(y_test_nw, y_pred_nw)
rmse_no_weather = np.sqrt(mean_squared_error(y_test_nw, y_pred_nw))

print("\n" + "="*60)
print("MODEL COMPARISON")
print("="*60)
print(f"  WITH weather:    MAE = {mae_xgb:.3f} €/MWh,  RMSE = {rmse_xgb:.3f}
print(f"  WITHOUT weather: MAE = {mae_no_weather:.3f} €/MWh,  RMSE = {rmse_r
print(f"  Difference:      MAE = {mae_no_weather - mae_xgb:+.3f} €/MWh")
```

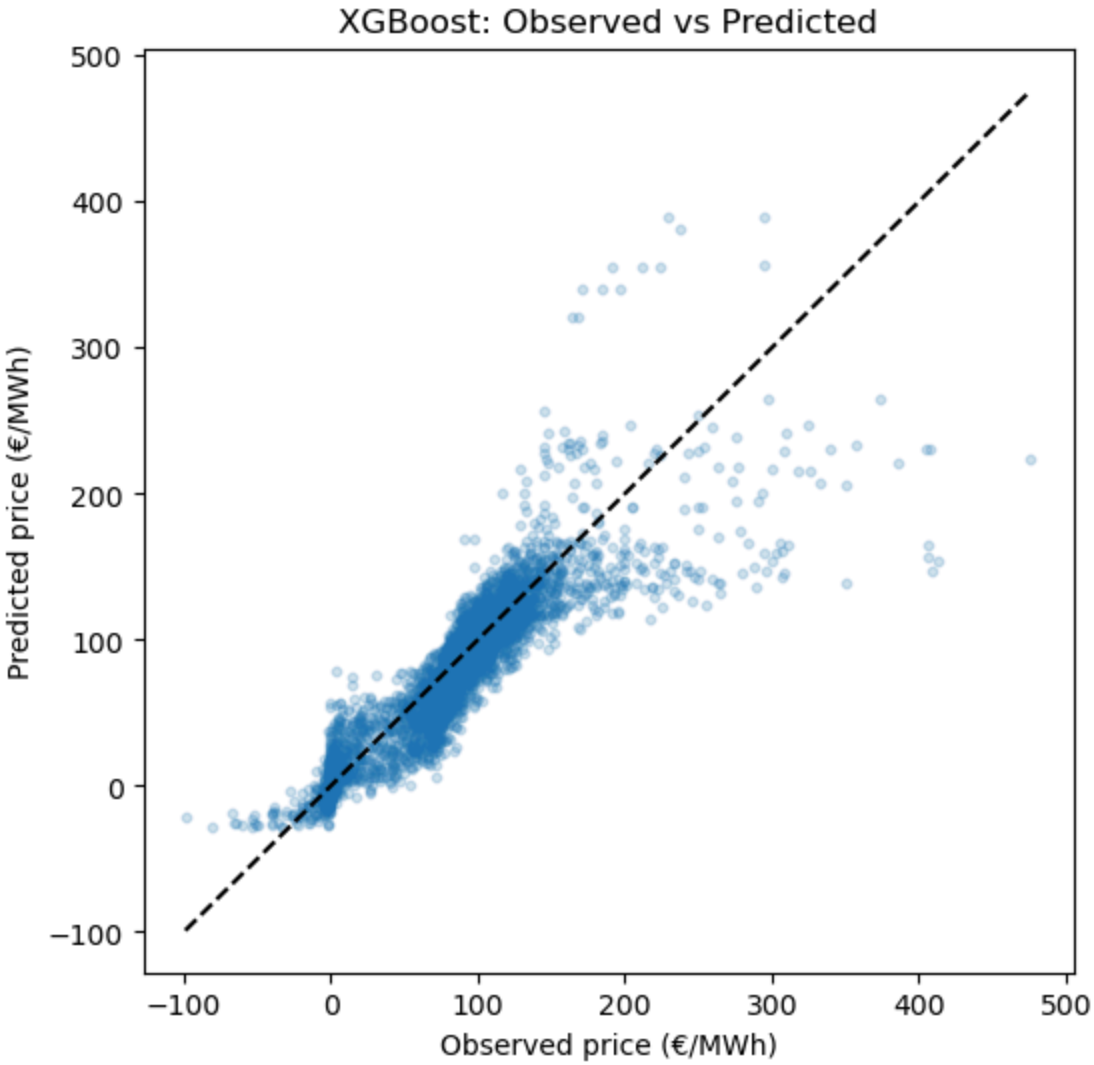
```
Fitting 5 folds for each of 162 candidates, totalling 810 fits

Best parameters (no weather): {'learning_rate': 0.01, 'max_depth': 2, 'min_c
hild_weight': 20, 'n_estimators': 500, 'subsample': 0.8}

============================================================
MODEL COMPARISON
============================================================
  WITH weather:    MAE = 14.058 €/MWh,  RMSE = 23.389 €/MWh
  WITHOUT weather: MAE = 14.422 €/MWh,  RMSE = 23.789 €/MWh
  Difference:      MAE = +0.364 €/MWh
```

Interpretation: The results indicate that including weather variables leads to a slight improvement in forecasting accuracy. However, since ENTSO-E generation forecasts already capture most weather-related price effects, the additional information from Open-Meteo is limited, and the benefit remains modest.

Interpretation of model performance

The model substantially outperforms both the naive baseline and SARIMA, demonstrating that the structural relationships between residual load, price persistence, and calendar effects remain predictive. The model with weather features is used for the final predictions.

In [36]:
```
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred_xgb, alpha=0.2, s=10)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--", color="black"
)
plt.xlabel("Observed price (€/MWh)")
plt.ylabel("Predicted price (€/MWh)")
plt.title("XGBoost: Observed vs Predicted")
plt.show()
```



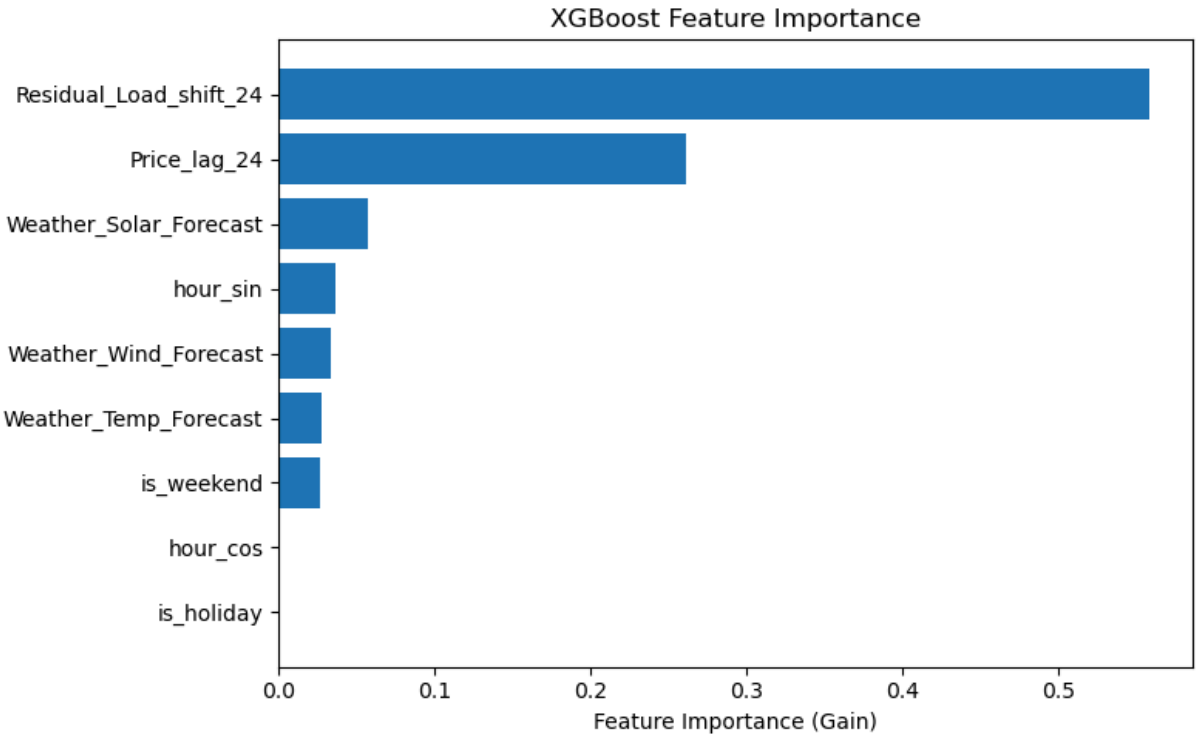
Plot summary: XGBoost achieves tighter predictions around the diagonal, particularly in the low-to-mid price range (0–150 €/MWh). While linear regression shows systematic underprediction at higher prices, XGBoost reduces this bias somewhat, though both models still underpredict extreme price spikes.

Feature importance analysis

XGBoost provides a built-in measure of feature importance based on how often each feature is used in tree splits and how much it contributes to reducing the prediction error.

In [37]:
```
importance = best_model.feature_importances_
feature_names = X.columns

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importance
}).sort_values("Importance", ascending=True)

plt.figure(figsize=(8, 5))
```

```
plt.barh(importance_df["Feature"], importance_df["Importance"])
plt.xlabel("Feature Importance (Gain)")
plt.title("XGBoost Feature Importance")
plt.tight_layout()
plt.show()
```



Interpretation of Feature Importance:

The chart shows the feature importance of an XGBoost model measured by Gain, which reflects how much each feature contributes to improving the model's splits.

Key observations:

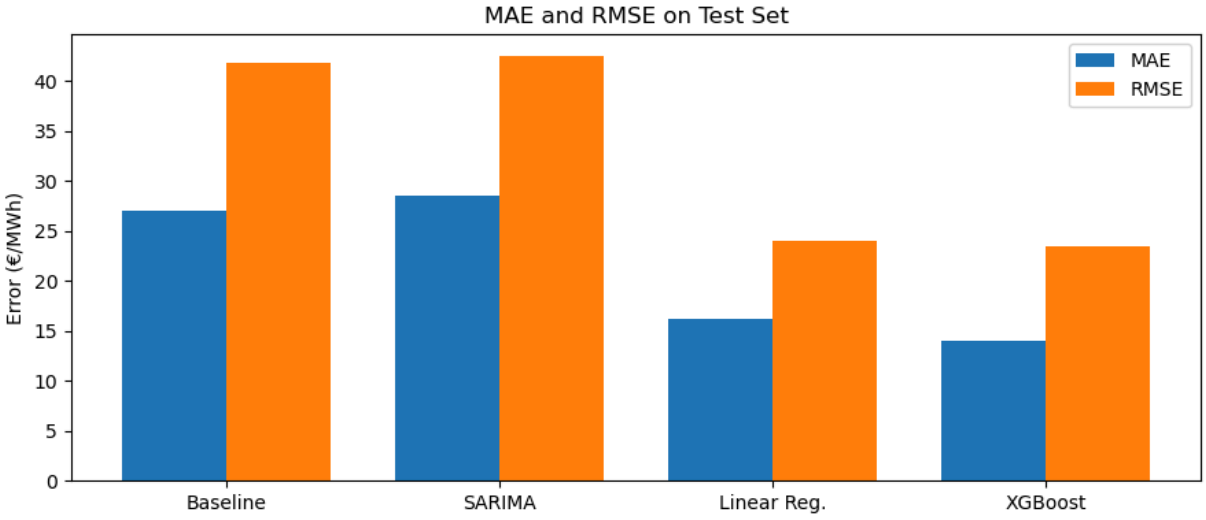
- Residual Load dominates the prediction, confirming its central role in price formation.
- The cyclical hour features (hour_sin, hour_cos) jointly capture intraday price patterns arising from demand cycles and solar generation.
- Weather variables make moderate contributions, potentially capturing information not fully reflected in the ENTSO-E generation forecasts.
- The binary time features is_weekend and is_holiday have low importance, likely because their influence on prices is already captured through residual load (lower demand on weekends and holidays reduces residual load, which in turn affects prices).

Part 4. Conclusion

4.1 Results

The figure below compares MAE and RMSE across our four forecasting models on the test set.

In [38]:
```
models = ["Baseline", "SARIMA", "Linear Reg.", "XGBoost"]
mae = [mae_base, mae_sar, mae_lr, mae_xgb]
rmse = [rmse_base, rmse_sar, rmse_lr, rmse_xgb]

x = np.arange(len(models))
w = 0.38

plt.figure(figsize=(9, 4))
plt.bar(x - w/2, mae, width=w, label="MAE")
plt.bar(x + w/2, rmse, width=w, label="RMSE")
plt.xticks(x, models)
plt.ylabel("Error (€/MWh)")
plt.title("MAE and RMSE on Test Set")
plt.legend()
plt.tight_layout()
plt.show()
```



Plot summary: The naive baseline and SARIMA models exhibit similarly high error levels. Thus, SARIMA does not provide a meaningful improvement over a persistence forecast. Linear regression substantially reduces both MAE and RMSE, indicating a clear gain in average predictive accuracy. XGBoost provides a small but consistent improvement over linear regression, which indicates the presence of weak nonlinear effects. However, the majority of the performance gain seems to arise from incorporating relevant explanatory variables rather than from model complexity alone.

4.2 Predictions for January 27th, 2026

Using the trained models above, we generate price predictions for January 27, 2026. The prediction assumes we are on January 25 and have access to ENTSO-E forecasts for January 26.

In [ ]:

```
jan25_start = "2026-01-25 00:00:00+01:00"
jan25_end = "2026-01-25 23:00:00+01:00"

# XGBoost
X_jan25 = df_xgb.loc[jan25_start:jan25_end, features].copy()

predictions_jan26 = best_model.predict(X_jan25)

jan26_index = X_jan25.index + pd.Timedelta(hours=24)
results_jan26 = pd.DataFrame({
    "Timestamp_UTC": jan26_index,
    "Hour_Berlin": jan26_index.tz_convert("Europe/Berlin").hour,
    "Predicted_Price": predictions_jan26
})


hours_berlin = results_jan26["Hour_Berlin"]

# baseline prediction
pred_baseline = df.loc[jan25_start:jan25_end, "Price_DayAhead"].values

# linear regression
features_lr = ["Residual_Load_shift_24", "Price_lag_24", "is_weekend", "is_h
X_jan = df_lr.loc[jan25_start:jan25_end, features_lr].copy()

X_jan = X_jan.ffill().bfill()

X_scaled = pd.DataFrame(scaler.transform(X_jan), index=X_jan.index, columns=
pred_lr = lr_model.predict(sm.add_constant(X_scaled, has_constant='add')).va

# sarima
ts_sarima = df.loc[:jan25_end, "Price_DayAhead"]

ts_sarima = ts_sarima[~ts_sarima.index.duplicated(keep='first')]

ts_sarima = ts_sarima.asfreq('h').ffill()

sarima_full = SARIMAX(ts_sarima,
                      order=(2, 1, 2),
                      seasonal_order=(1, 0, 1, 24),
                     ).fit(disp=False)

pred_sarima = sarima_full.get_forecast(steps=24).predicted_mean.values


plt.figure(figsize=(14, 6))
plt.plot(hours_berlin, pred_baseline, marker='o', label="Naive Baseline", al
plt.plot(hours_berlin, pred_sarima, marker='o', label="SARIMA", alpha=0.7)
plt.plot(hours_berlin, pred_lr, marker='o', label="Linear Regression", linew
plt.plot(hours_berlin, predictions_jan26, marker='o', label="XGBoost", linew

plt.xlabel("Hour of Day (Europe/Berlin)")
plt.ylabel("Predicted Price (€/MWh)")
plt.title("Predicted Day-Ahead Prices for January 27, 2026 - Model Comparisc
plt.xticks(range(0, 24))
```

```
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Plot summary: while all models capture the daily price cycle, the naive baseline is the smoothest and least reactive. SARIMA produces moderate peaks, whereas linear regression and XGBoost respond more strongly to daytime and evening demand. Linear regression shows the highest peak. XGBoost follows a similar pattern but with slightly reduced extremes.

4.3 Potential improvements

The models perform well for typical price levels, but all of them struggle during extreme price spikes. This indicates that some important drivers of electricity prices are missing.

We tried to include additional variables such as gas prices and CO_2 prices. However, we could not find reliable and up-to-date data with the required hourly resolution and forecast availability. Therefore, these variables were not included in the final models.

All models in this study produce point forecasts only. In future work, probabilistic forecasting could be considered to better reflect uncertainty, especially during volatile periods.

Neural network–based models could also be a good fit. They may better capture nonlinear relationships and complex interactions between load, weather, and prices. However, they usually require more data and careful tuning and their interpretability is lower compared to linear models.

Finally, further feature engineering, such as separating peak and off-peak hours or adding interaction terms, could improve model performance.

In [ ]:
```
# ------ GENERATING PDF FILE ------
!python -m nbconvert --to webpdf --embed-images --allow-chromium-download 20
```